

请自选一种编译器的源代码进行阅读分析，在此基础上完成总体架构设计，然后在编译器实现的每个阶段完成相应部分的设计文档，格式及内容不限，每个阶段均需写出编码之前的设计，编码完成之后对设计的修改情况。文档包括如下内容：

1. 参考编译器介绍：总结所阅读的编译器的总体结构、接口设计、文件组织等内容
2. 编译器总体设计：介绍自己的将要实现的编译器的总体结构、接口设计、文件组织等内容
3. 词法分析设计：编码前的设计、编码完成之后的修改。若用大模型辅助生成代码，则需写出对生成代码的评价和对代码的人工修改
4. 语法分析设计：编码前的设计、编码完成之后的修改。若用大模型辅助生成代码，则需写出对生成代码的评价和对代码的人工修改
5. 语义分析设计：编码前的设计、编码完成之后的修改。若用大模型辅助生成代码，则需写出对生成代码的评价和对代码的人工修改
6. 代码生成设计：编码前的设计、编码完成之后的修改。若用大模型辅助生成代码，则需写出对生成代码的评价和对代码的人工修改
7. 代码优化设计：编码前的设计、编码完成之后的修改。若用大模型辅助生成代码，则需写出对生成代码的评价和对代码的人工修改。未选择 MIPS 代码生成的同学无需完成此项内容

参考编译器介绍

目前为止还没有参考编译器

编译器总体设计

总体结构

首先我做的是CPP的MIPS编译器

我想的是,既然这个符号表管理和错误处理贯穿整个编译过程的始终,然后又有五个主要逻辑流程

我就先使用一个manager类来管理符号表和错误处理,并且持有这两个类

然后manager还持有Lexer ,Parser等词法分析器,语法分析器

然后整个编译过程都由这个manager调度完成

每个像Lexer,Parser这样的中间处理类都持有errorController和符号表的引用

在需要注册符号和错误时使用引用来注册

然后串行的处理输入

值得注意的是,我写的时候没有参考指导书上的内容,是先完整地读取了源程序,然后再对其进行处理

而不是使用指导书中的流式数据

接口设计

如前所述,我使用一个manager类来管理整个的编译数据流,然后这个manager类持有子流程和错误处理和符号表

所有的编译接口都是调用这个manager的public方法,然后这个manager再去调用子流程的方法

我不想使用任何的全局变量,因此像语法树,符号表这样的全局的量都是manager持有的

文件组织

目前来说我打算每个子功能分一个文件夹,然后每个文件夹里面只有h和cpp

然后等到我把整个编译器做完了,我就给他分成frontend midend,backend还有manager

然后main.cpp放在外面

词法分析设计

总体设计

刚开始的总体设计非常简单,就是设计一个类,分词,然后给每个词输出一个token.type和token.str

具体怎么做就可以类似于像我写的快读一样:

```
1  int read1() {
2      int x = 0, w = 1;
3      char ch = 0;
4      while (!isdigit(ch)) {
5          if (ch == '-') w = -1;
6          ch = getchar();
7      }
8      while (isdigit(ch)) {
9          x = x * 10 + (ch - '0');
10         ch = getchar();
11     }
12     return w * x;
13 }
```

和语法分析的状态机很类似,一直读,读到不是

但是词法分析会更简单,只凭借一点的前缀就可以判断token类型,再使用专门的switch来获取token

关于大模型?

我使用了kimi来教我cpp,因为很久没写了不太会了

然后还使用kimi来生成了一些def文件和注释,因为这个如果要手敲到文件里面就太机械且没有教育意义了

就是类似于这样的:

```
1  X(LEX_INVALID_TOKEN,      'a', "invalid token")
2  X(SEM_IDENT_REDEF,        'b', "ident redefined")
3  X(SEM_IDENT_UNDEF,        'c', "ident undefined")
4  X(SEM_FN_PARAM_NUM,       'd', "function parameter number mismatch")
5  X(SEM_TYPE_MISMATCH,      'e', "type mismatch")
6  X(SEM_FN_RET_TYPE,        'f', "function should not return value")
7  X(SEM_FN_RET_MISS,        'g', "function missing return value")
8  X(SEM_CONST_ASSIGN,       'h', "constant assignment")
9  X(SYN_MISS_SEMICN,        'i', "missing semicolon")
10 X(SYN_MISS_PARENT,        'j', "missing parenthesis")
```

```

11 X(SYN_MISS_BRACK,      'k', "missing bracket")
12 X(SEM_PRINTF_MISMATCH, 'l', "printf format mismatch")
13 X(SEM_BREAK_CONTINUE, 'm', "break or continue outside loop or switch")
14 X(SYN_CASE_DUP,        'n', "duplicated case or default in switch")

```

其他的所有代码都是我手写的,我敢保证这一点

写代码的时候的一些小巧思

首先是最喜欢的这个双符号运算符判断

```

1  LexTokenKind Lexer::get_operator_id(){
2      ui c1=get_cur();
3      ui c2=get_next();
4      ui key = (c1<<8 | c2);
5      isDoubleOperator=true;
6      switch (key) {
7          case (('=' << 8) | '='): return EQL;    //优雅,永不过时
8          case (('<' << 8) | '='): return LEQ;
9          case (('>' << 8) | '='): return GEQ;
10         case (('!' << 8) | '='): return NEQ;
11         case (('&' << 8) | '&'): return AND;
12         case (('|' << 8) | '|'): return OR;
13     }
14     isDoubleOperator=false;
15     switch (c1)
16     {
17         case '+': return PLUS;
18         case '-': return MINU;
19         case '*': return MULT;
20         case '/': return DIV;
21         case '%': return MOD;
22         case '<': return LSS;
23         case '>': return GRE;

```

使用位运算来使得switch可以适配双运算符(ui是unsigned int)

然后是这个使用X宏来简化代码书写,并且统一代码:

```

1  const char *LextokenKindName(LexTokenKind kind) {
2      #define X(name,num) [num]=#name,
3      static const char * tokenName[]={
4          #include "../config/lexer_token.def"
5      };
6      #undef X
7      if(kind<0||kind>=LTK_COUNT) return "Error";
8      return tokenName[kind];
9  }
10

```

然后就是避免转义错误,使用一个通用的get_char来读取字符串

如果遇到\就再读一位

```
1 void Lexer::read_one_char(){
2     int ch = get_cur();
3     if (ch == EOF) return;
4     token.push_back(ch);
5     move_forward();
6     if (ch == '\\') {
7         int escaped = get_cur();
8         if (escaped == EOF) return;
9
10        token.push_back(escaped);
11        move_forward();
12    }
13 }
```

最终整个类只有一个主要的public处理方法(如果不算读取和debug输出的话):

```
1 void Lexer::scan(){
2     while (1)
3     {
4         move_until_nbc();
5         if(curPos>=source.length()) break;
6         int ch=get_cur();
7         int nt=get_next();
8         reset_token();
9         if(ch=='"'){//STRCON
10            tokenList.push_back(get_const_str_token());
11        }else if(ch=='\'){//CHARCON
12            tokenList.push_back(get_const_char_token());
13        }else if(isdigit(ch)){ //INTCON
14            tokenList.push_back(get_const_int_token());
15        }else if(isalpha(ch)||ch=='_'){ //Ident or KW
16            tokenList.push_back(get_ident_or_revserve_token());
17        }else if(ch=='/'&& nt=='/') { //line comment
18            skip_line();
19        }else if(ch=='/'&& nt=='*') { //block comment
20            skip_block();
21        }else{ //OPERA
22            tokenList.push_back(get_operator_token());
23        }
24    }
25 }
26 }
```

语法分析设计

总体设计

首先是看见原来的这个结构有一个不太合理的地方

就是这个Parser需要获取Lexer.h来获取LexToken的定义,很明显这个是非常不必要的

于是乎我就写了两个h文件,分别存放所有的enum和所有的struct定义

这样就可以解耦了

然后就是关于语法分析本身的设计

首先,题目要求的输出方式其实就是对于语法树的后序遍历的输出

既然都要这样输出了,我就考虑直接建一个语法树,然后遍历这个树来进行输出

那么就要考虑这个语法树应当如何建立

语法树的建立

首先,是这个语法节点的存储问题,考虑到可用性和工程化,我使用unique_ptr来做这个节点的存储,并使用专门的分配函数

```
1 ParToken * Parser::make_ParToken(ParTokenKind kind){
2     parTokenPool.push_back(make_unique<struct ParToken>());
3     ParToken * node = parTokenPool.back().get();
4     node->type=kind;
5     return node;
6 }
```

这样每次新建节点时,只要调用这个make就行了

然后这些节点会随着Parser类的释放而释放,是内存安全的

其次,是这个语法树的建立问题,我没有使用Optional的那些语法特性,就只是给每个语法成分写了一个方法:

```
1 // 常量声明 ConstDecl → 'const' BType ConstDef { ',' ConstDef } ';' // i
2 ParToken * Parser::parse_ConstDecl(){
3     ParToken * node = make_ParToken(ConstDecl);
4     match(node, CONSTTK);
5     node->add(parse_BType());
6     node->add(parse_ConstDef());
7     while (check(COMMA))
8     {
9         match(node, COMMA);
10        node->add(parse_ConstDef());
11    }
12    match(node, SEMICN);
13    return node;
14 }
```

范式就是:

1. 新建语法节点
2. 检查可选成分
3. 挂载叶子节点来匹配字符(match本身包含挂载逻辑)
4. 挂载子树节点来匹配语法成分

5. 返回语法节点(这样就可以递归构造树了)

然后关于语法成分的检测,分为一下几个情况

- 是必要的字符,使用match匹配,没有则报错
- 是可选的语法成分或字符,使用if
- 是可以重复多次的成分或字符,使用while
- 对于语法成分,使用专门的is_xxxxx_ahed函数处理,这个函数通过匹配共同前缀后的字符来判断语法成分

最后在后序遍历语法树即可:

```
1 void CompilerManager::debug_report_parser(){
2     if(errorController.has_error()){
3         report_error();
4     }
5     report_parser_tool(parTreeRoot);
6 }
7
8 void CompilerManager::report_parser_tool(ParToken * node){
9
10    if(node->type!=Leaf) {
11        for (auto & child : node->childs)
12        {
13            report_parser_tool(child);
14        }
15        //除了<BlockItem>, <Decl>, <BType> 之外
16        if(node->type!=BlockItem&&node->type!=Decl&&node->type!=BType)
17            fprintf(out, "%s\n", ParTokenKindName(node->type));
18    }else{
19
20        fprintf(out, "%s %s\n", LexTokenKindName(node->lexToken->tokenType)
21            ,node->lexToken->raw_string.c_str());
22    }
23
24 }
```

工具函数

首先是一个小巧思

```
1 int Parser::get_prev_line(){
2     if(curPos<=0) return 1;
3     return lexTokenList[curPos-1].line_num;
4 }
5 //这个东西是用来错误处理的,因为错误看的行其实是上一行
```

然后就是比较一般的函数

```
1 const LexToken & Parser::get_cur(){
```

```

2     return get_offset_next(0);
3 }
4 const LexToken & Parser::get_next(){
5     return get_offset_next(1);
6 }
7 const LexToken & Parser::get_offset_next(int offset){
8     static const LexToken eofToken{Eof, string(), 0}; //这里是因为返回引用,所以需要有一个静态变量
9     if(curPos>=0&&curPos+offset<lexTokenList.size()){ //本身位置就不可以违法,所以>=0
10         return lexTokenList[curPos+offset];
11     }
12     return eofToken;
13 }
14 void Parser::move_backward(){
15     if(curPos>0) curPos--;
16 }
17 void Parser::move_forward(){
18     if(curPos<lexTokenList.size()) curPos++;
19 }
20 void Parser::match(ParToken * father, LexTokenKind tar){
21     if(get_cur().tokenType==tar){
22         ParToken * node =make_ParToken(Leaf);
23         node->lexToken=&get_cur();
24         father->add(node);
25         move_forward();
26     }else if(get_cur().tokenType==LexError){
27         move_forward();
28     }else {
29         errorController->register_ijk(tar,get_prev_line());
30     }
31 }
32
33 bool Parser::check(LexTokenKind tar,int offset=0){
34     if(tar==get_offset_next(offset).tokenType) return true;
35     return false;
36 }
37 bool Parser::check_any(initializer_list<LexTokenKind> list,int offset=0){
38     for(auto & kind :list){
39         if(get_offset_next(offset).tokenType==kind) return true;
40     }
41     return false;
42 }

```

check只检查不消耗,match检查并消耗,不匹配则报错